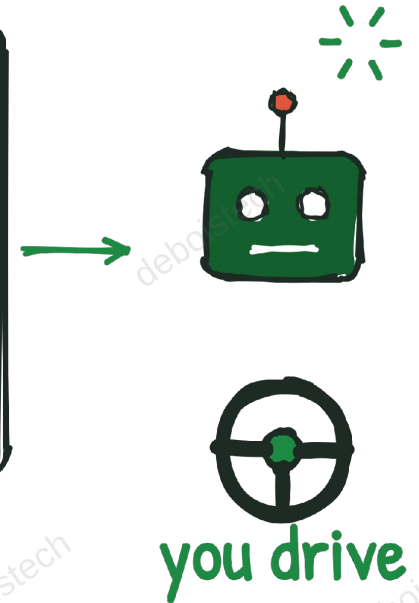
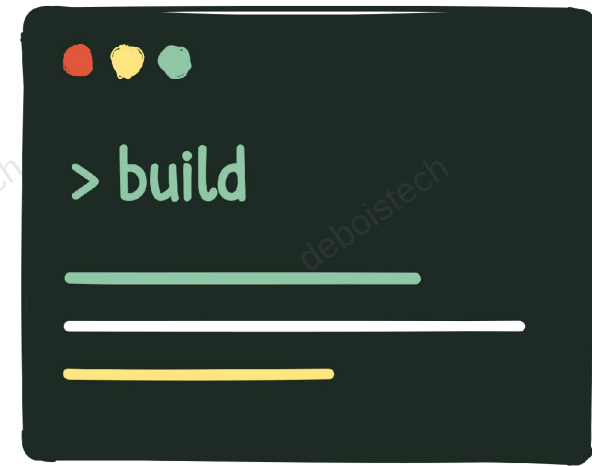


AI Coding Techniques

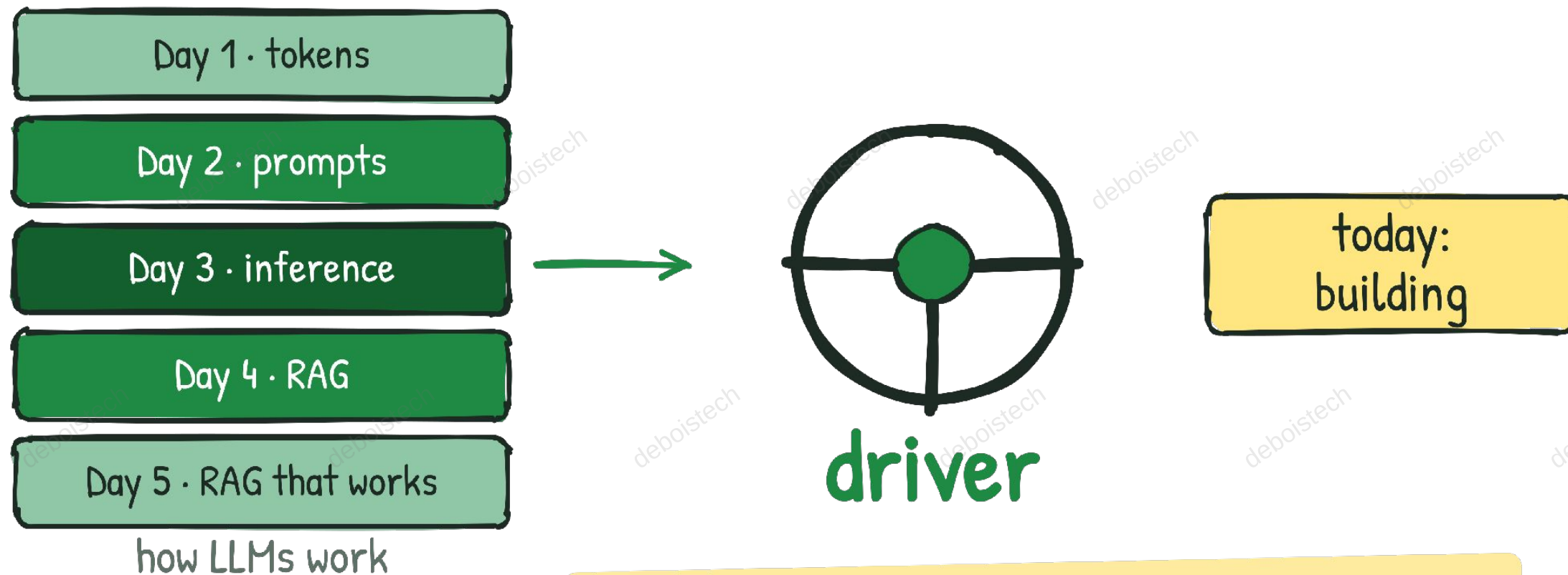
From understanding LLMs to building with them



DAY 6 · ASTRA · TRACK 1, WEEK 1

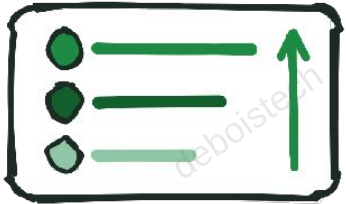
The hook

Under the hood, then building.



The landscape

Four tools, one defining trait each.



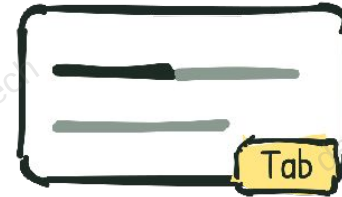
Antigravity

agents
in parallel



Cursor

AI-native
editor



Copilot

lives in the
editor you use



Claude Code

agent in
the terminal

not deep dives: just enough to recognize them

Comparison at a glance

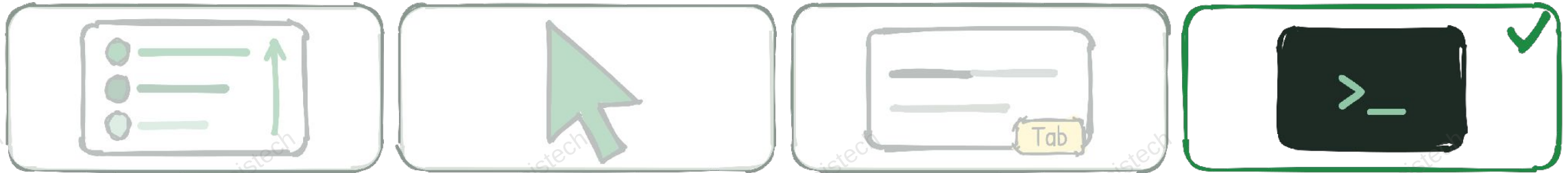
Strength and typical use, one line each.

	strength	typical use
Antigravity	parallel agents + artifacts	delegate big tasks
Cursor	Tab + multi-file agent	daily AI-heavy editing
Copilot	fits any editor, GitHub-native	everyday completions, PRs
Claude Code	terminal agent, no lock-in	codebase-wide tasks, automation

a reasoned choice, not an arbitrary one

The choice

One tool for execution, not all four.



all six features

in one tool



the loop is visible

in the terminal



ideas transfer

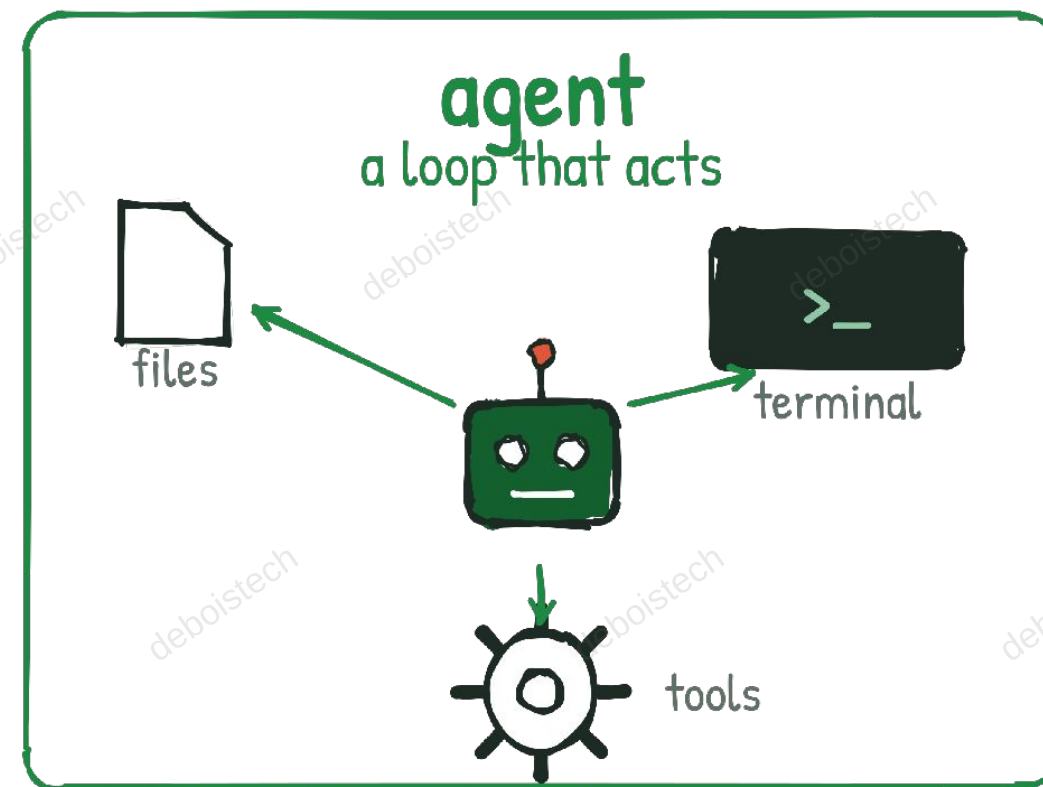
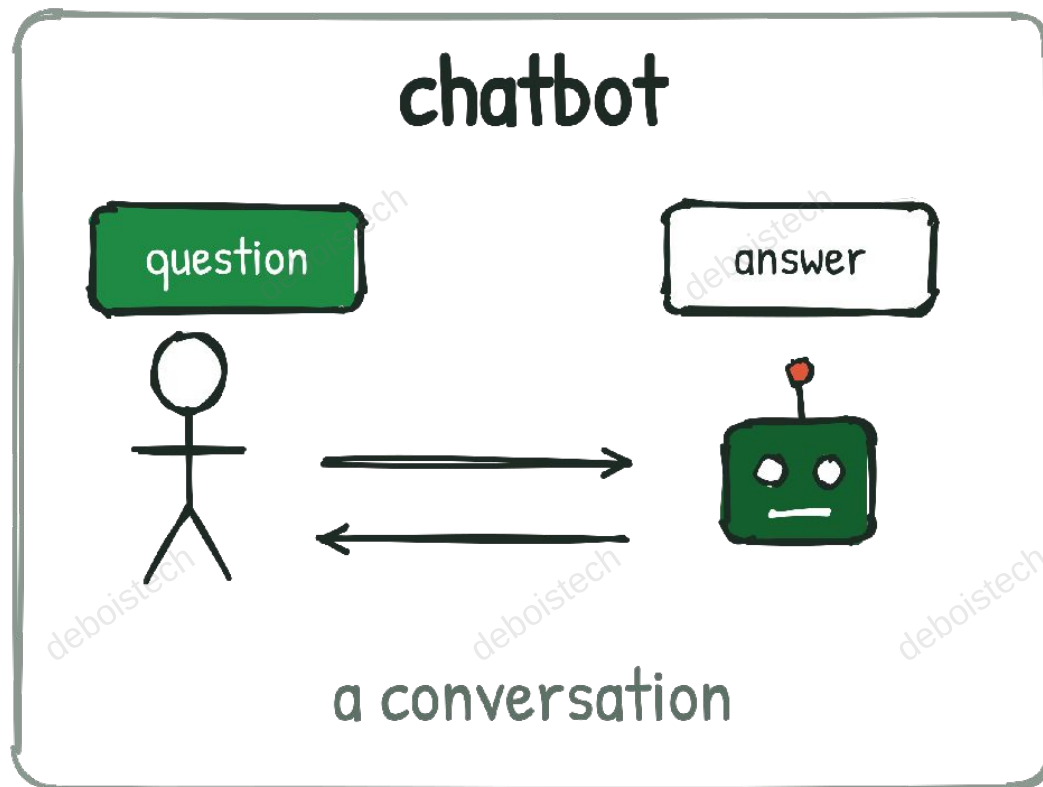
to the other three



tour = the landscape. deep dive = what is possible.

Isn't this just a smarter chatbot?

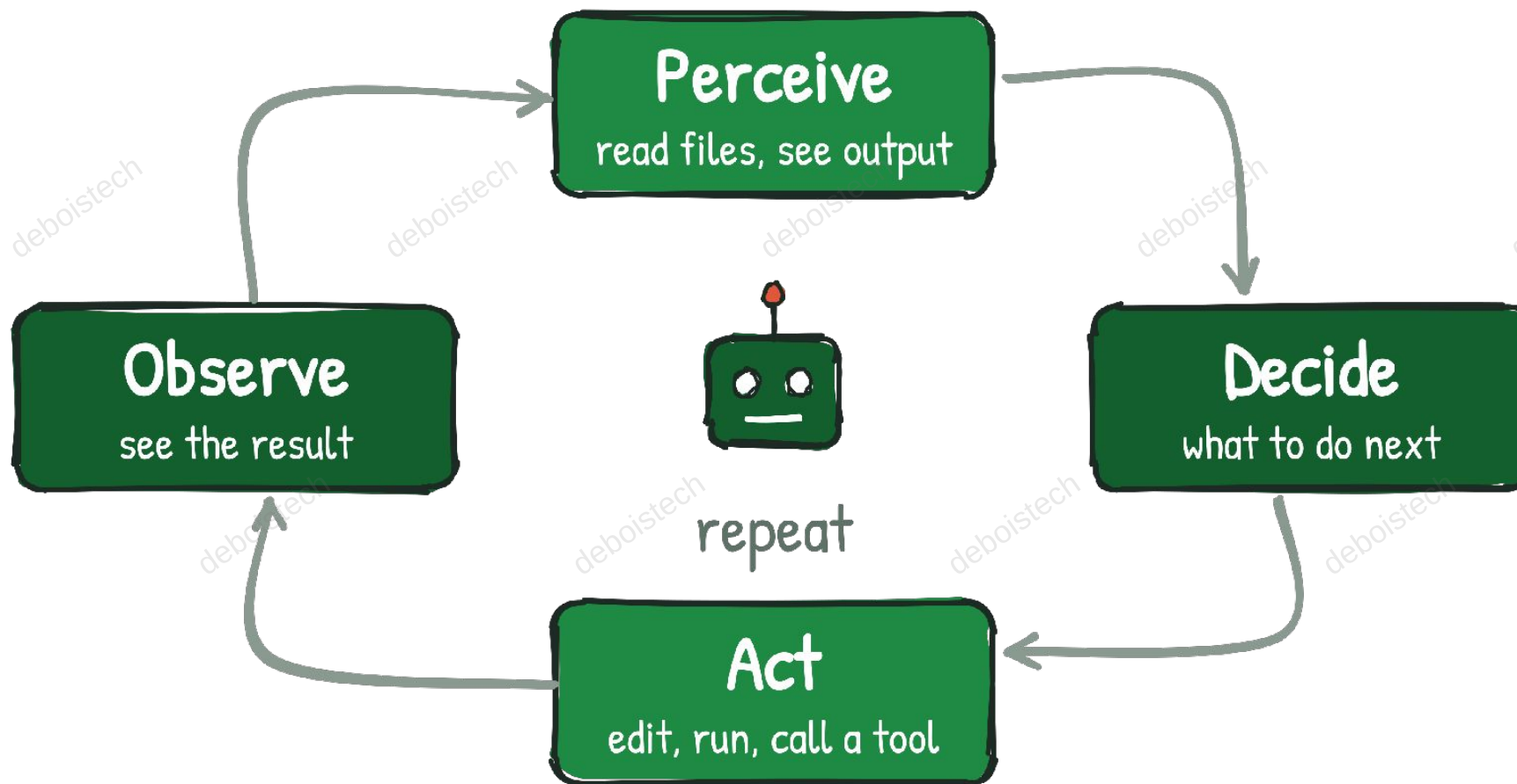
No: a model wired into a loop.



a model wired into a loop

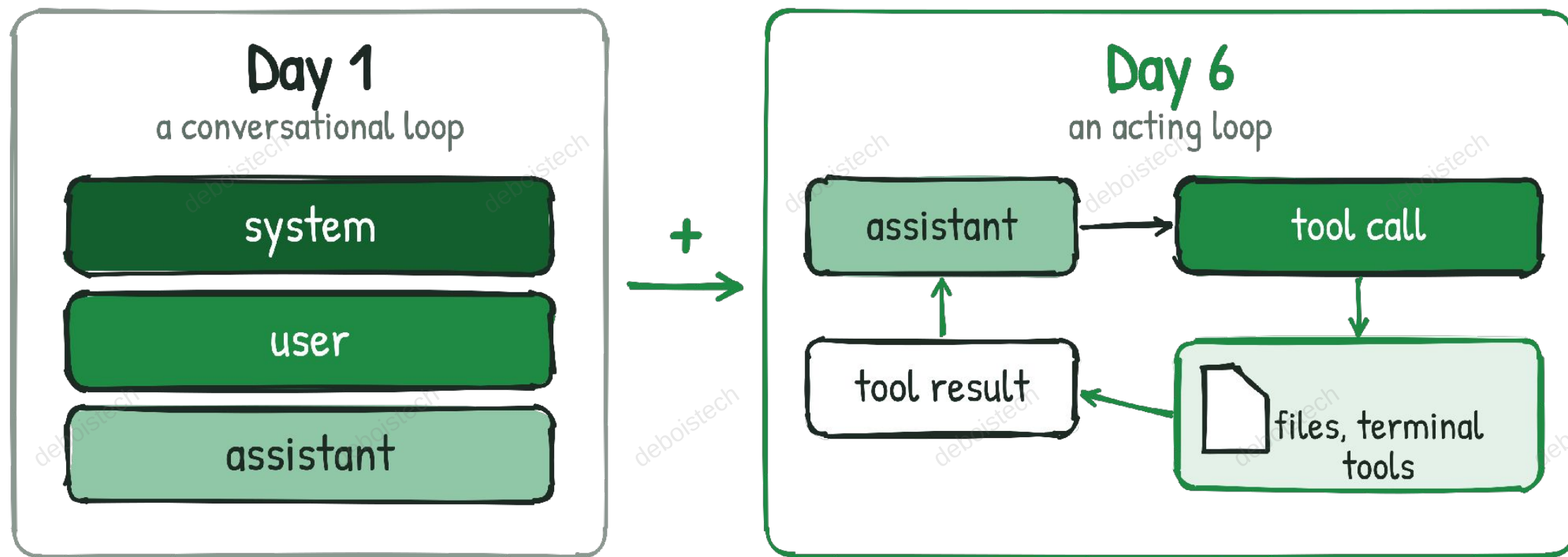
The agent harness loop

Perceive, decide, act, observe. Repeat.



What this extends

The environment joins the conversation.



the environment becomes part of the chat

Model + harness = agent

The model proposes. The harness does.



the model

- reads the context
- proposes the next step

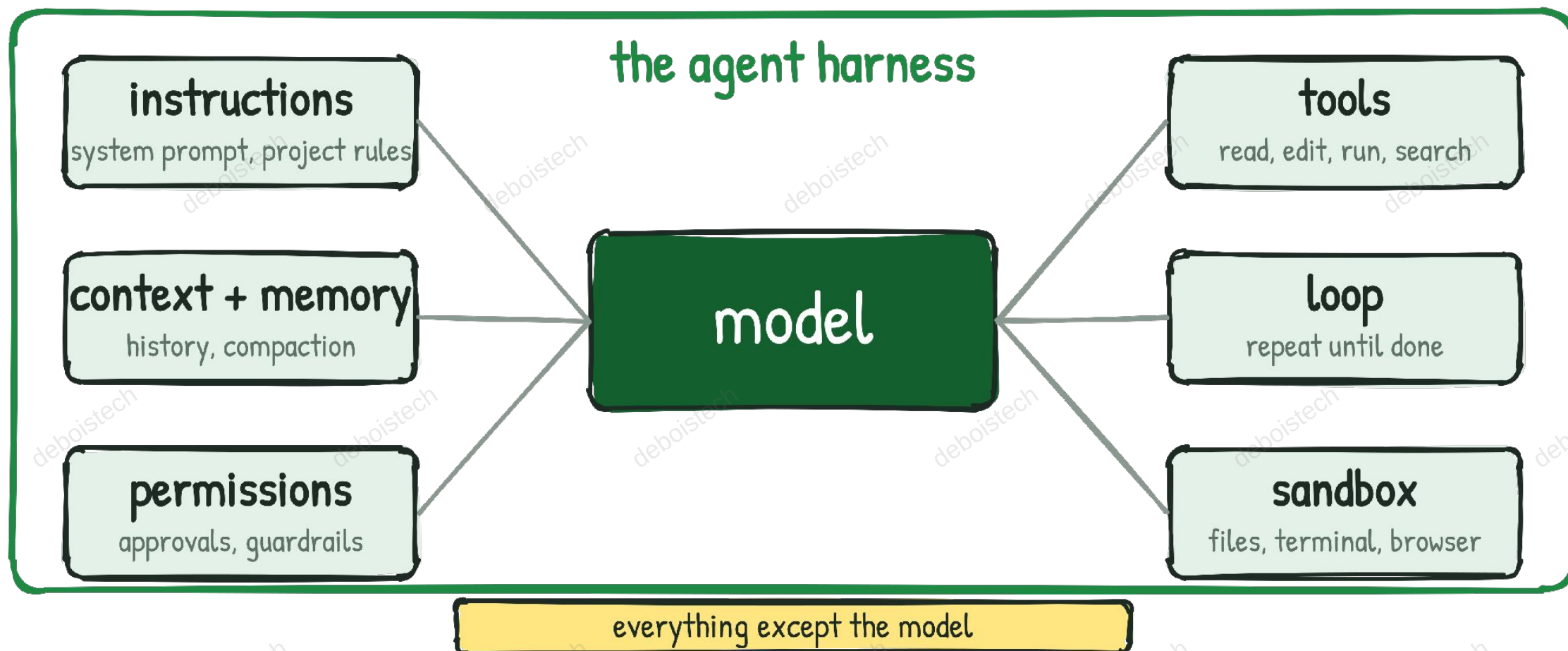
the harness

- runs the tools
- enforces the rules
- manages the context

the model proposes. the harness executes.

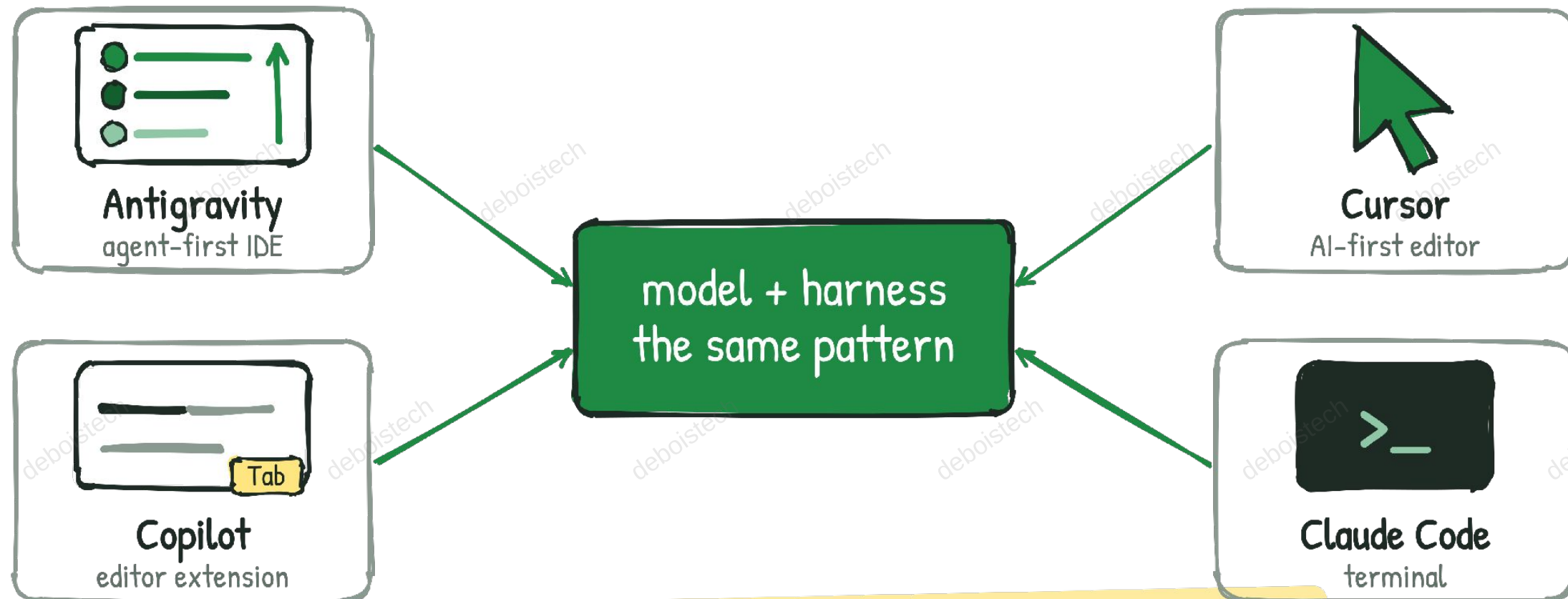
What comprises a harness

Everything around the model.



Four tools, one pattern

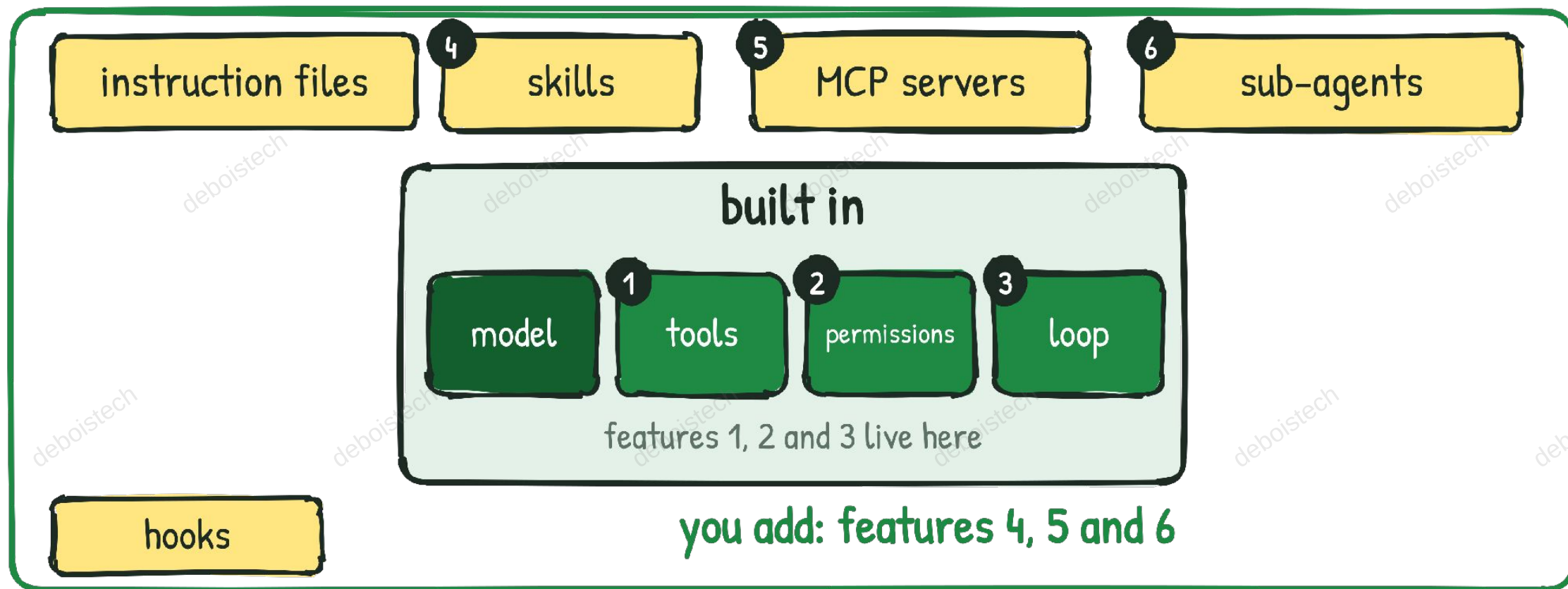
Same architecture, different surface.



different surface, same pattern

The harness you add

The tool ships one. You extend it.

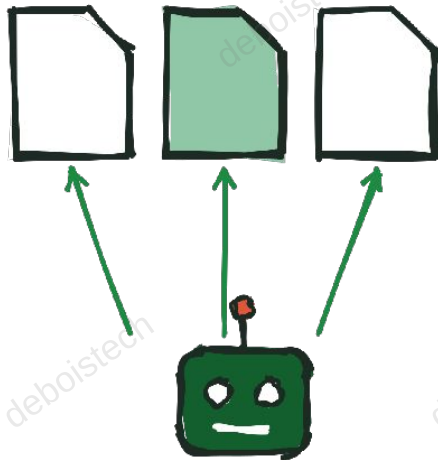


Feature 1: agentic file editing

It reads and edits real files, not just answers.



live: watch it in the project



edits across files



you decide

accept

reject

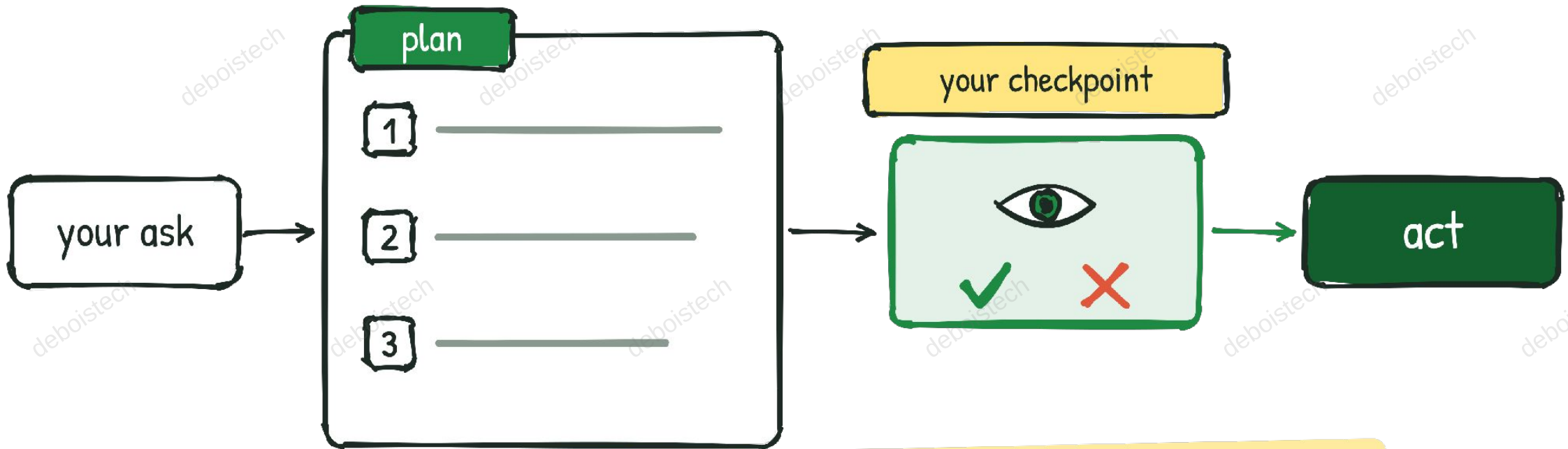
review the diff, not just that it ran

Feature 2: plan mode

It proposes a plan before it acts.



Live: watch it in the project



if you would not approve it, do not let it run

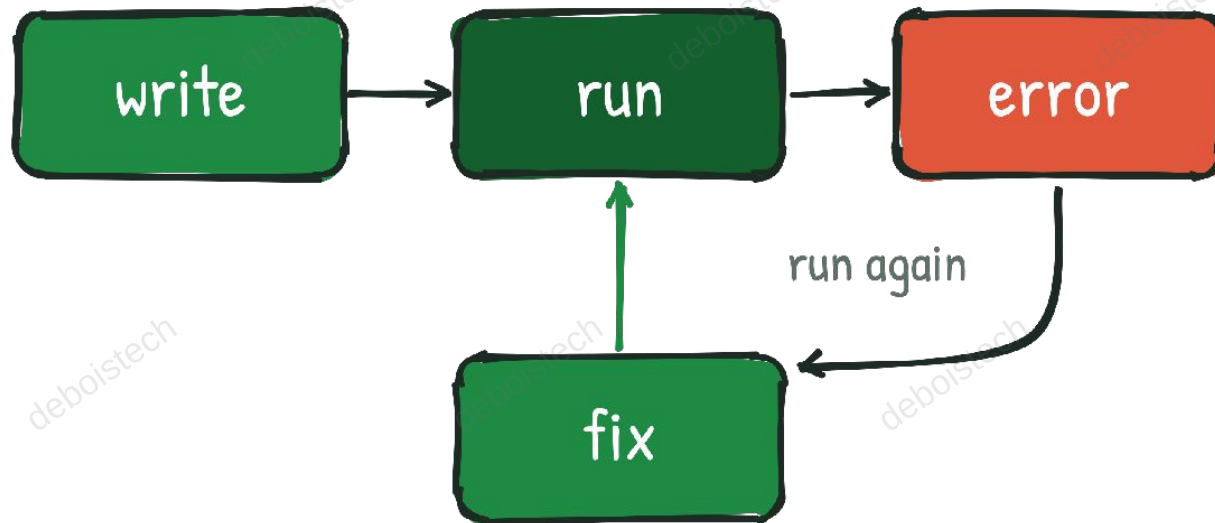
Feature 3: self-correcting on errors

It runs the code, sees errors, fixes them.



live: watch it in the project

does it converge?



same error again and again?



step in

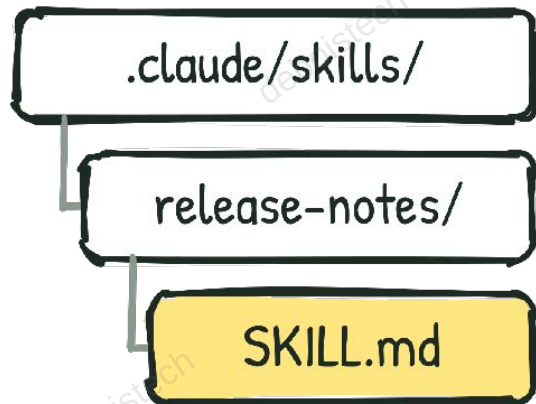
self-correction is not infallible

Feature 4: skills

Teach it once, reuse it every time.



live: watch it in the project



write it once

name: release-notes
description: for release notes

steps

- 1 read the git log
- 2 group by type
- 3 write in house style

the agent matches
your ask to this

then follows
these steps

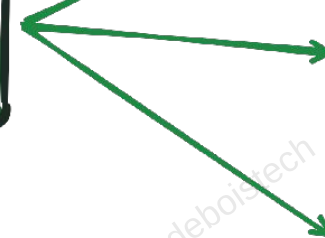
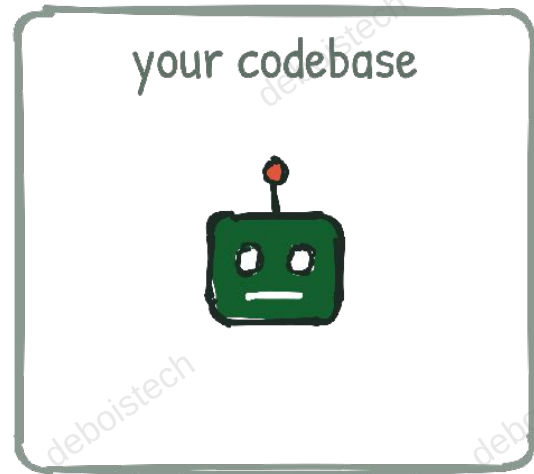
a skill is still instructions: read them

Feature 5: MCP

The agent reaches outside the codebase.



trust boundary



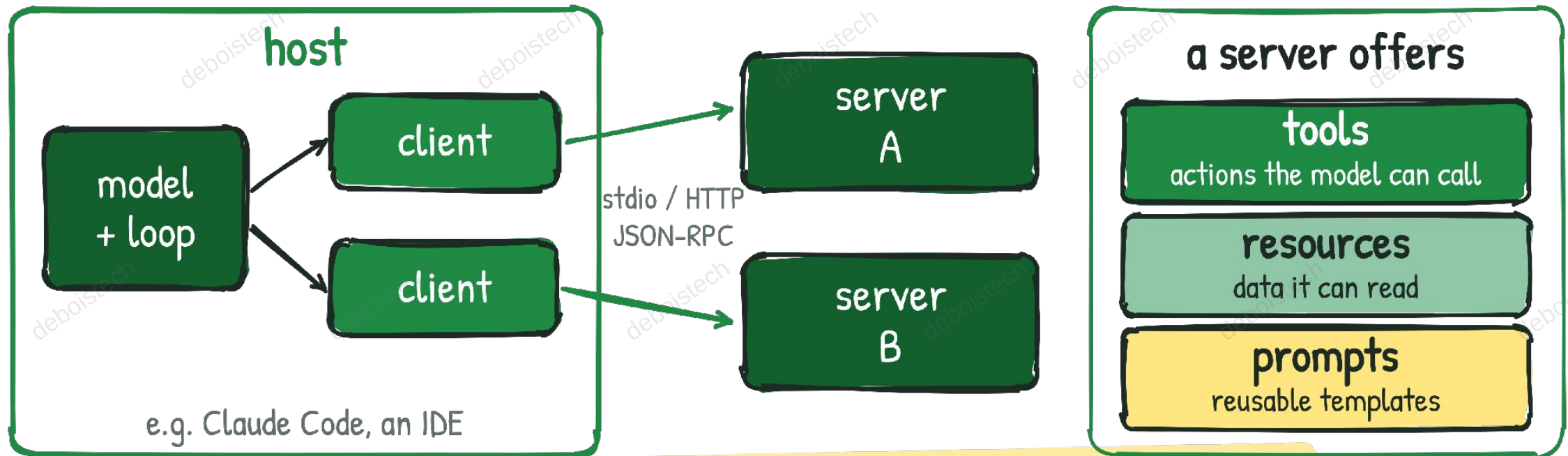
live: watch it in the project

Day 2: injection, more surface

know exactly what it can reach

MCP: the components

Host, client, server, and what a server offers.



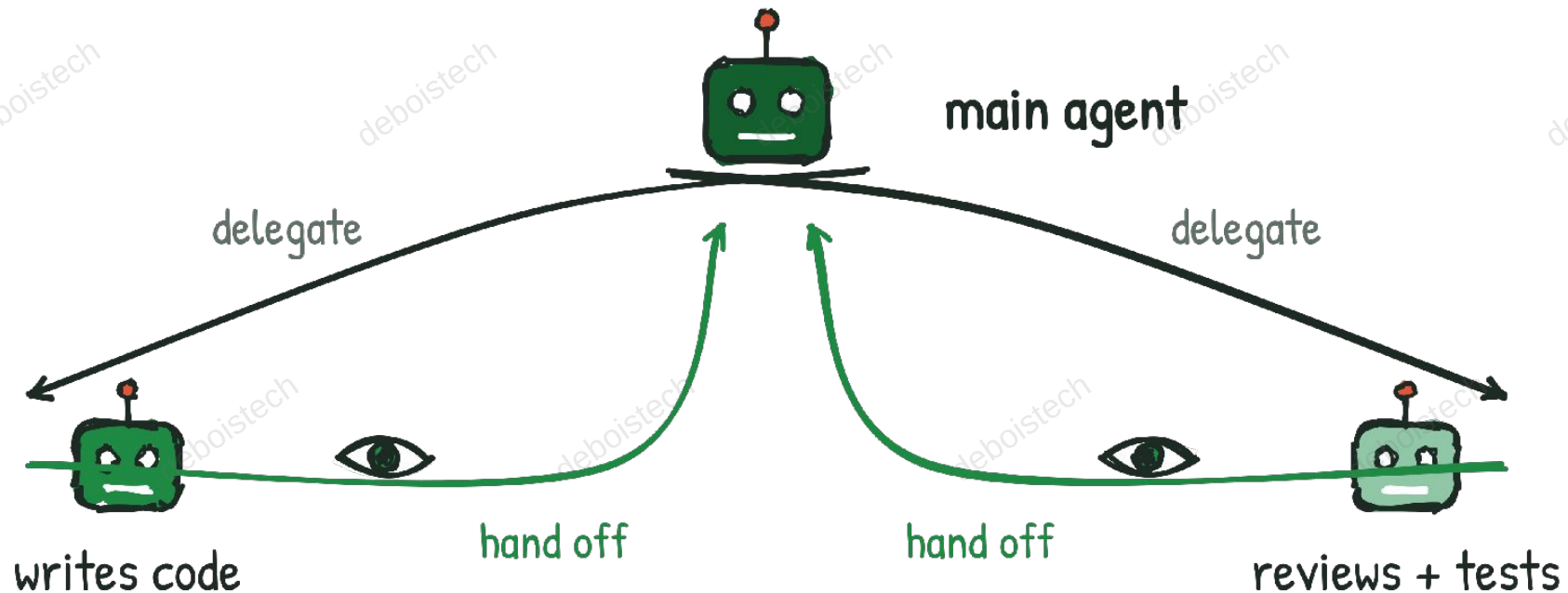
host owns the loop. server owns the capability.

Feature 6: sub-agents

It delegates a sub-task to another agent.

①—②—③—④—⑤—⑥ **sub-agents**

live: watch it in the project



delegation is not exemption from review

The best-practice checklist

Not blind trust: know exactly where to look.



review
diffs



review
plans



doubt
self-fixes



read the
skill



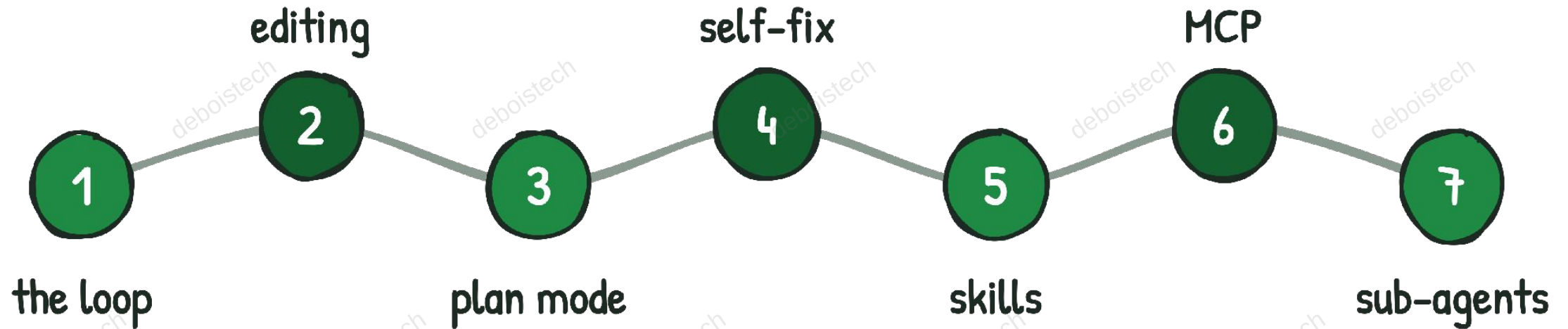
know the
MCP reach



check
sub-agents

Recap: what you just watched

Seven ideas, each with a place to pause.



each one: a capability.
each one: a place to pause and check.

Up next: Week 2

Today you used agents. Next you build them.



Week 2 · from Day 7

Build the agent

agent basics, multi-agent orchestration,
and everything inside that loop

